State Estimation and Filtering for Robotic Perception

AA273

RGB front end, Back End Optimizers, Pose Graph Optimization vs Bundle Adjustment, Loop Closure



Stanford University

MSL

AA273: State Estimation and Filtering for Robotic Perception

Announcements:

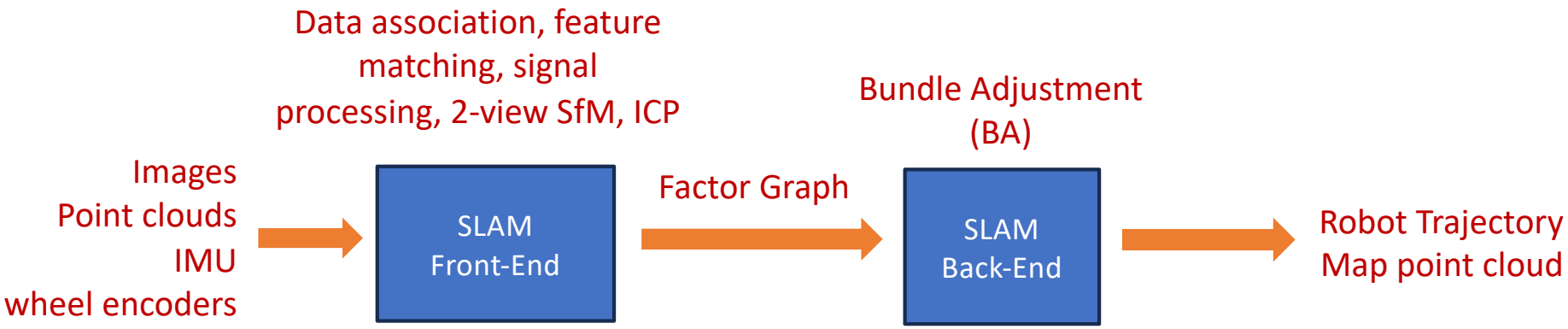
- HW 6 due Fri Mar 6
- Project Reports: due Tue Mar 17
  - 5-6 pages + 1 page per group member > 1
  - Latex/Overleaf, IEEE conf format
  - >= 10 refs
  - Describe research and results
  - Re-use proposal material
- Project Presentations: 3/10, 3/12
  - 4 min pres + 1 min Q and A
  - Sign up for preferred time slot
  - Share slides through zoom meeting
  - Present on your computer

Lecture 17:

- RGB SLAM front end (high level)
- Back end optimizers
- Bundle adjustment vs pose graph optimization
- Loop closure

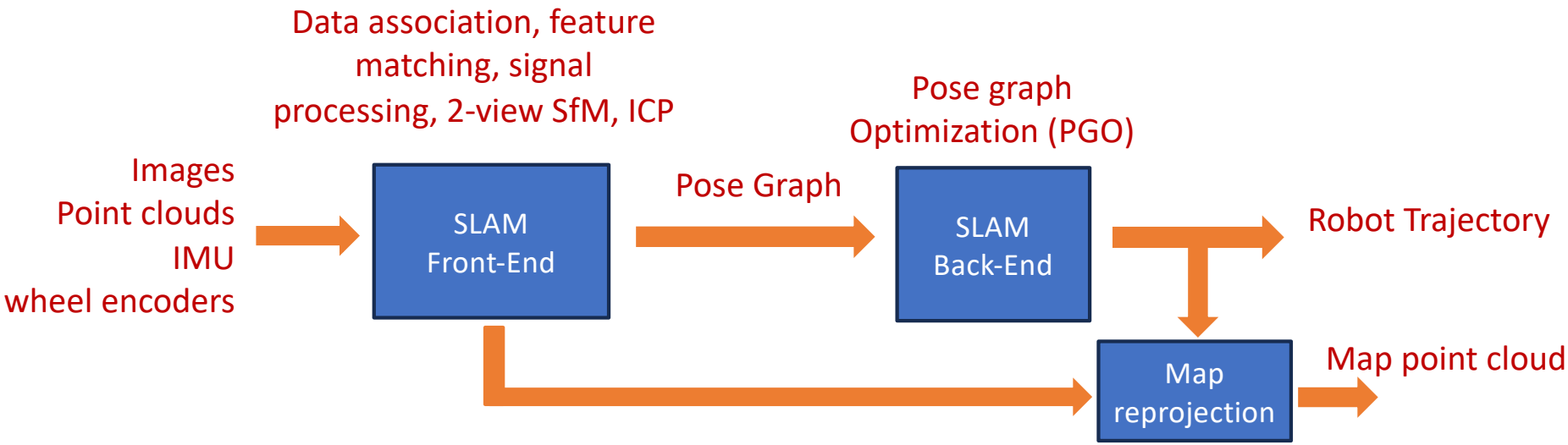
Factor Graphs: A Unifying Perspective for SLAM

(Frank Dellaert, 2012, 2017)



Bundle Adjustment SLAM - Produces both robot traj and map point cloud

Factor Graphs: A Unifying Perspective for SLAM



Pose Graph SLAM - Produces robot traj first, map obtained in post processing.

NeRFBridge for SLAM with NeRFs

NerfBridge: Bringing Real-time, Online Neural Radiance Field Training to Robotics

Javier Yu, Jun En Low, Keiko Nagami, and Mac Schwager

Stanford University

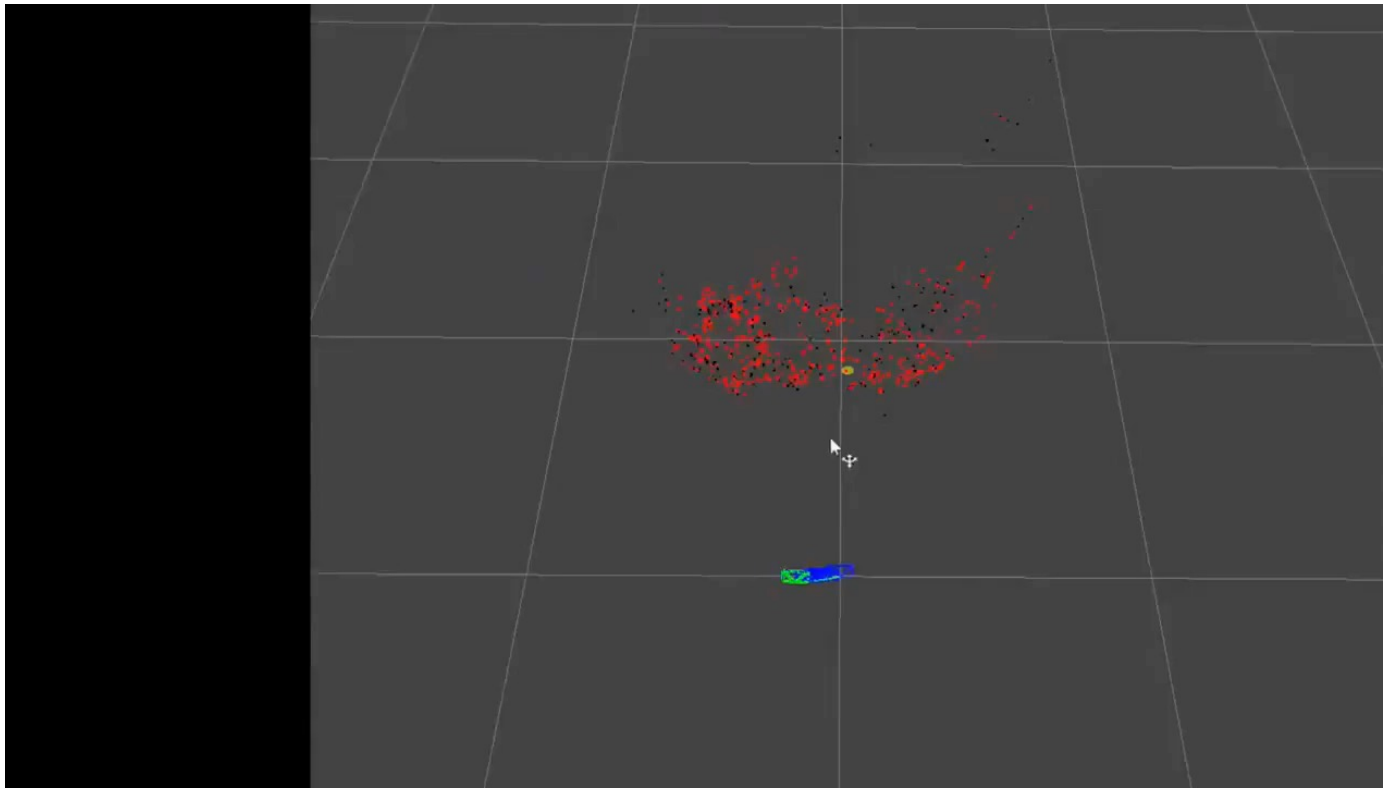
Contact: javieryu@stanford.edu
Code: https://github.com/javieryu/nerf_bridge



SLAM in ROS NAV2 toolbox



SLAM in Space for Asteroid Reconstruction



Iterative Closest Point (ICP): iterate with closest point heuristic

- For each point in P_B

  p_i^B match to closest point p_j^A from P_A

- Solve for (τ_{AB}, R_{AB}) with Wahba's prob

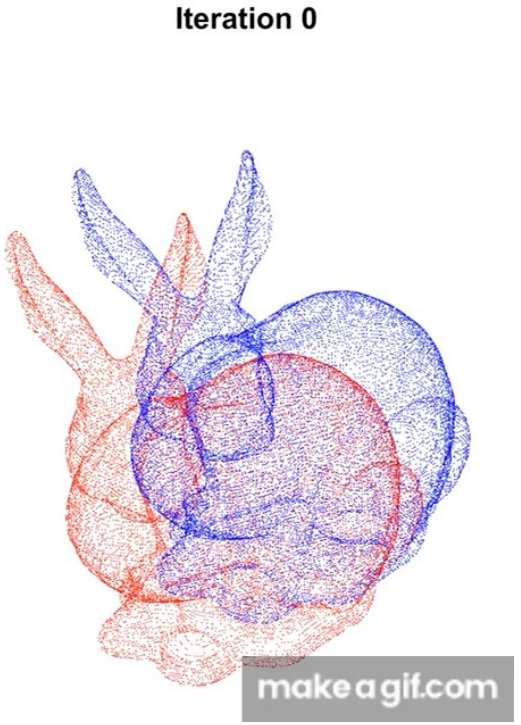
Center: $\tilde{p}_i^A = p_i^A - \bar{p}^A, \quad \tilde{p}_i^B = p_i^B - \bar{p}^B$

SVD: $\sum_i \tilde{p}_i^B (\tilde{p}_i^A)^T = U \Sigma V^T$

Rotation: $R_{AB} = V U^T$

Translation: $\tau_{AB} = \bar{p}^A - R_{AB} \bar{p}^B$

- Repeat until convergence



Reduction to Wahba's Problem, SVD Solution

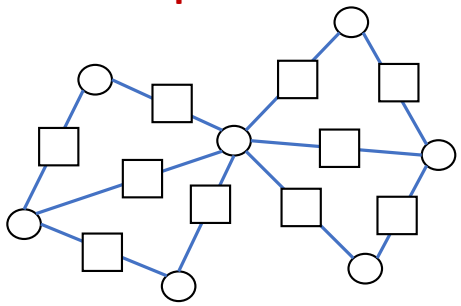
On board last time.

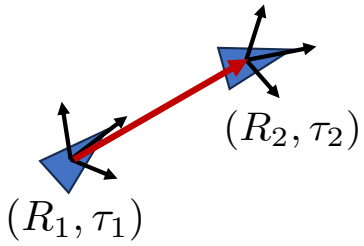
Hint: Schur's inequality: For any square matrix, the sum of the magnitudes of the eigenvalues <= sum of the singular values

$$\sum_i |\lambda_i| \leq \sum_i \sigma_i$$

Factors: Lidar and Odometry measurements

Factor Graph:



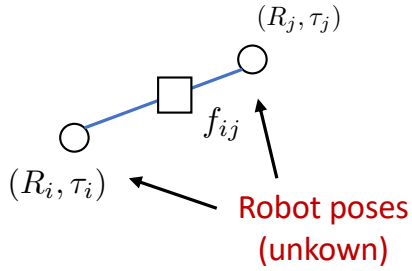


Odometry measurement: $(\hat{R}_{ij}, \hat{\tau}_{ij})$

$$\hat{R}_{ij} = R_i^T R_j + v_{ij}^R$$
$$\nu_{ij}^R \sim \mathcal{N}(0, \Sigma_{ij}^R)$$

$$\hat{\tau}_{ij} = R_i^T (\tau_j - \tau_i) + \nu_{ij}^\tau$$
$$\nu_{ij}^\tau \sim \mathcal{N}(0, \Sigma_{ij}^\tau)$$

Odometry factor



Odometry factor:

$$f_{ij} = \lambda_{ij}^R \| R_i \hat{R}_{ij} - R_j \|^2 + \lambda_{ij}^\tau \| R_i \hat{\tau}_{ij} - (\tau_j - \tau_i) \|^2$$

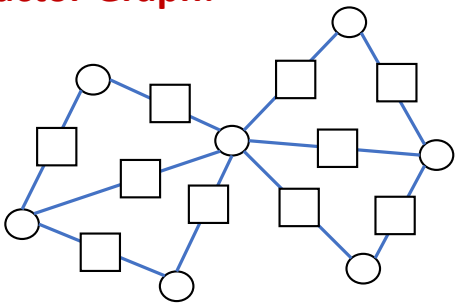
$$\lambda_{ij}^R = \det(\Sigma_{ij}^R)^{-1} \qquad \lambda_{ij}^\tau = \det(\Sigma_{ij}^\tau)^{-1}$$

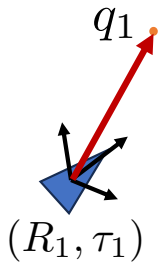
Weighting constants

Interpretation: Cost of *inconsistency* with measurement.

Factors: RGB-D measurements

Factor Graph:





RGB-D measurement:

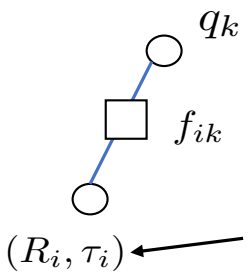
$$\hat{q}_{ik} = R_i^T(q_k - \tau_i) + \nu_{ik}^D$$

3D feature location in robot's frame (measured)

noise

$$\nu_{ik}^D \sim \mathcal{N}(0, \Sigma_{ik}^D)$$

RGB-D factor



3D feature location in global frame (unknown)

Robot pose (unkown)

RGB-D factor:

$$f_{ik} = \lambda_{ik}^D \| R_i \hat{q}_{ik} - (q_k - \tau_i) \|^2$$

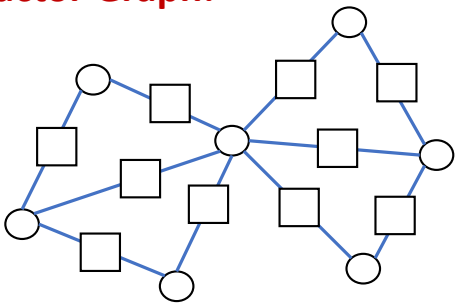
Weighting constant

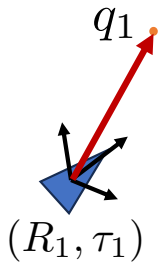
$$\lambda_{ik}^D = \det(\Sigma_{ik}^D)^{-1}$$

Interpretation: Cost of *inconsistency* with measurement.

Factors: RGB measurements

Factor Graph:





Camera calibration matrix

noise $\nu_{ik}^V \sim \mathcal{N}(0, \Sigma_{ik}^V)$

RGB measurement:

$$p_{ik}^h = K R_i^T (q_k - \tau_i) + \nu_{ik}$$

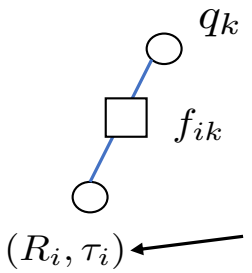
Homogeneous pixel coords

$$p_{ik}^h = (u_{ik}, v_{ik}, 1) z_{ik}^h$$

Measured pixel coords in image

Unknown pixel depth

RGB-D factor



3D feature location in global frame (unknown)

Robot pose (unkown)

RGB factor:

$$f_{ik} = \lambda_{ik}^V \| R_i K^{-1} p_{ik}^h - (q_k - \tau_i) \|^2$$

Weighting constant

$$\lambda_{ik}^V = \det(\Sigma_{ik}^V)^{-1}$$

Interpretation: Cost of *inconsistency* with measurement.

Pinhole camera model

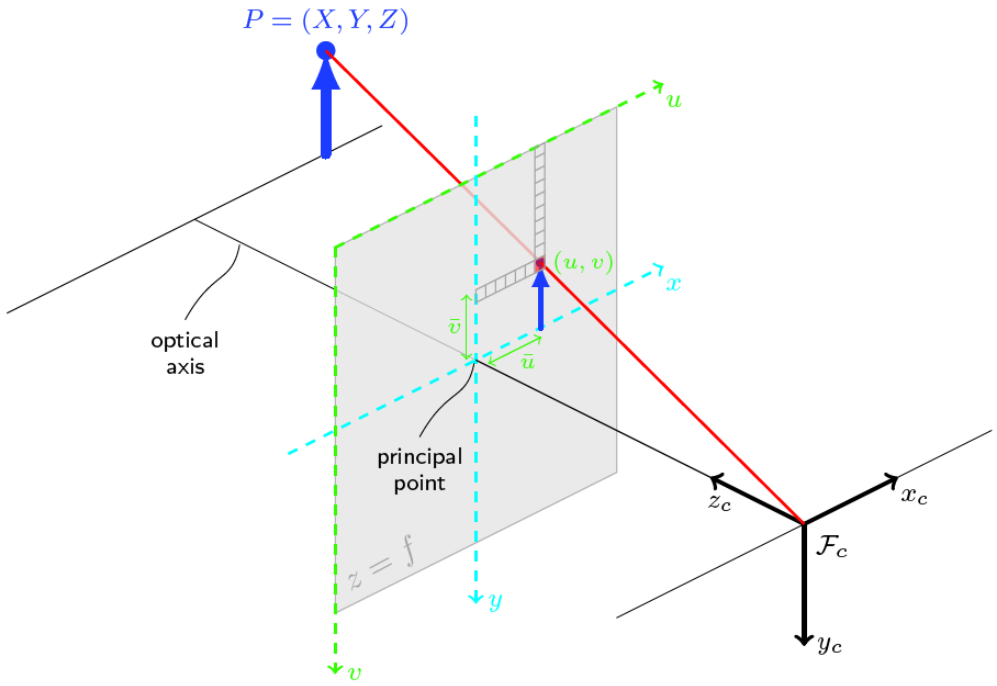
$$p^h = p_z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

$$K = \begin{bmatrix} \alpha & 0 & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$p^h = K[R \quad t]P_W^h$$

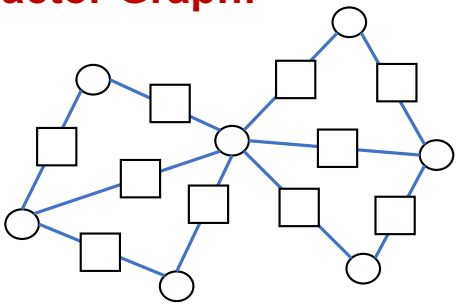
Intrinsic parameters

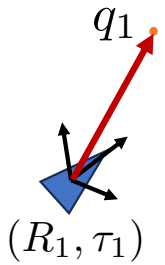
Extrinsic parameters



Factors: RGB measurements revisited

Factor Graph:





Camera calibration matrix

noise $\nu_{ik}^V \sim \mathcal{N}(0, \Sigma_{ik}^V)$

RGB measurement:

$$p_{ik}^h = K R_i^T (q_k - \tau_i) + \nu_{ik}$$

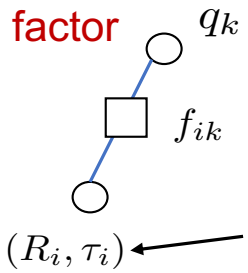
Homogeneous pixel coords

$$p_{ik}^h = (u_{ik}, v_{ik}, 1) z_{ik}^h$$

Measured pixel coords in image

Unknown pixel depth

RGB-D factor



3D feature location in global frame (unknown)

Robot pose (unkown)

RGB factor:

$$f_{ik} = \lambda_{ik}^V \| R_i K^{-1} p_{ik}^h - (q_k - \tau_i) \|^2$$

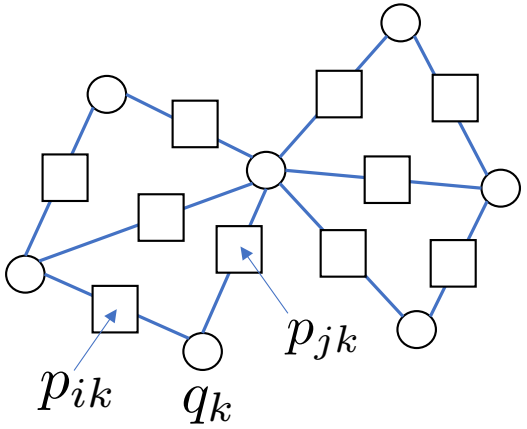
Weighting constant

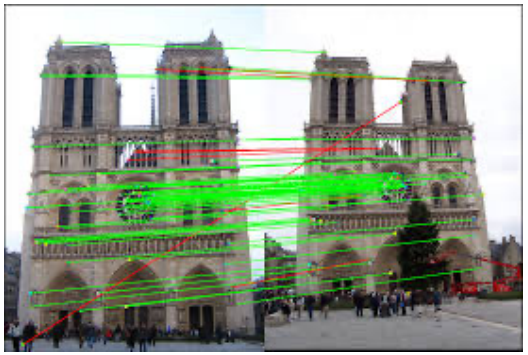
$$\lambda_{ik}^V = \det(\Sigma_{ik}^V)^{-1}$$

Interpretation: Cost of *inconsistency* with measurement.

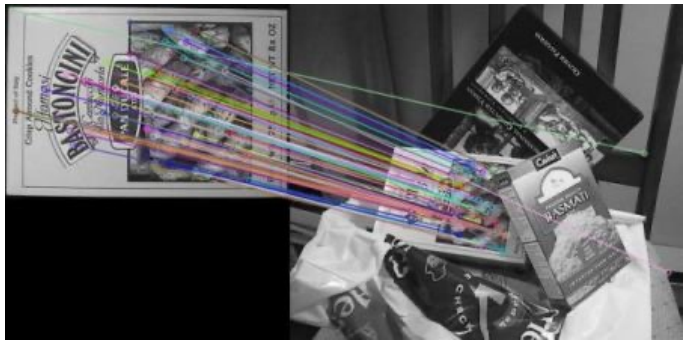
Assumes feature matched across images

Factor Graph:

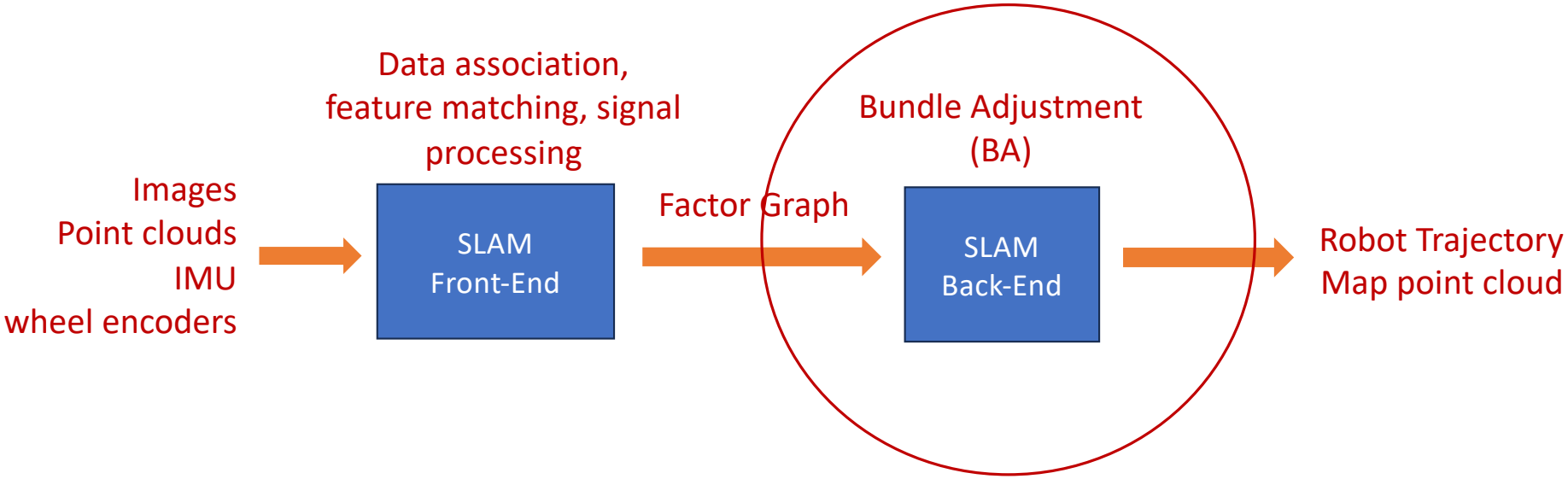






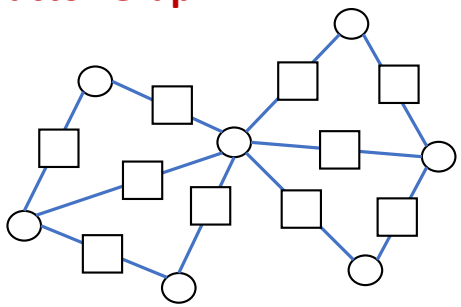


Factor Graphs: Bundle Adjustment



Bundle Adjustment Optimization

Factor Graph:



Sum of all factors gives total cost:

Odometry factors (LIDAR, IMU/encoders)

$$J((R_i,\tau_i),q_k) = \sum_{(\hat{R}_{ij},\hat{\tau}_{ij})} (\lambda_{ij}^R \|R_i\hat{R}_{ij} - R_j\|^2 + \lambda_{ij}^\tau \|R_i\hat{\tau}_{ij} - (\tau_j - \tau_i)\|^2)$$

$$+ \sum_{\hat{q}_{ik}} \underbrace{\lambda_{ik}^D \|R_i\hat{q}_{ik} - (q_k - \tau_i)\|^2}_{\text{RGB-D factors}} + \sum_{(u_{ik},v_{ik})} \underbrace{\lambda_{ik}^V \|R_i K^{-1} p_{ik}^h - (q_k - \tau_i)\|^2}_{\text{RGB factors}}$$

Optimization Problem (Bundle Adjustment) :

$$\min_{(R_i,\tau_i),q_k} J((R_i,\tau_i),q_k)$$

$$\text{subject to} \quad \left.\begin{aligned} R_i^T R_i = I \\ \det(R_i) = 1 \end{aligned}\right\} \; SO(3)$$

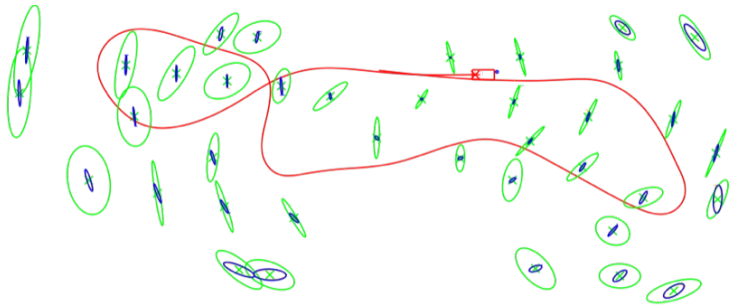
Interpretation: Find poses and feature locations that are *as consistent as possible* with all the measurements.

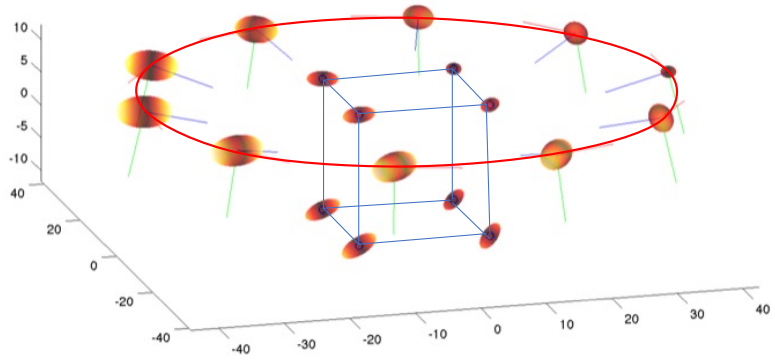
SLAM pose and map estimates:

$$\{(R_i^*,\tau_i^*),q_k^*\} = \arg\min_{(R_i,\tau_i),q_k} J((R_i,\tau_i),q_k)$$

GTSAM: The Standard Open-Source Factor Graph Optimizer

Frank Dellaert, and collaborators, GA Tech

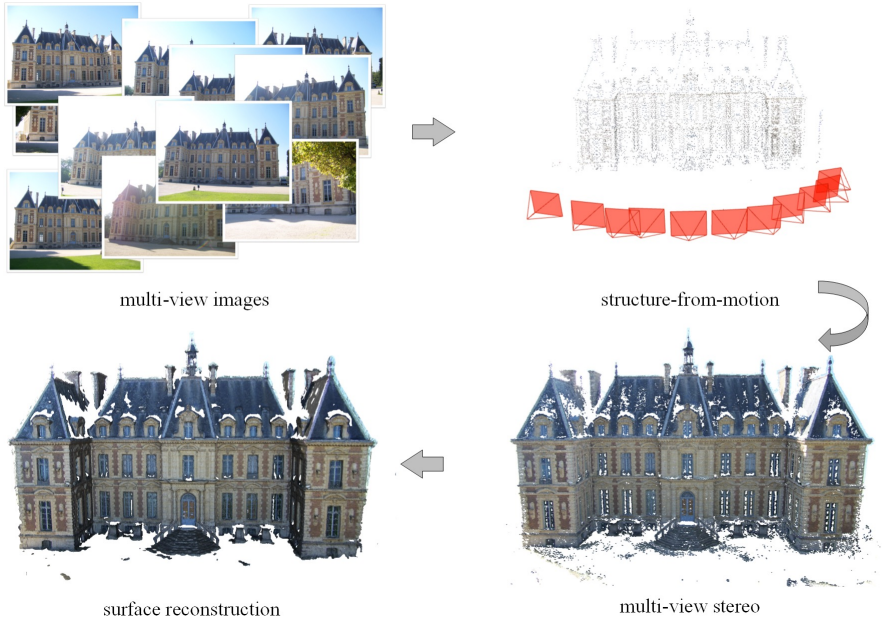




https://gtsam.org/

Colmap: The Standard Open-Source CV 3D Photogrammetry Package

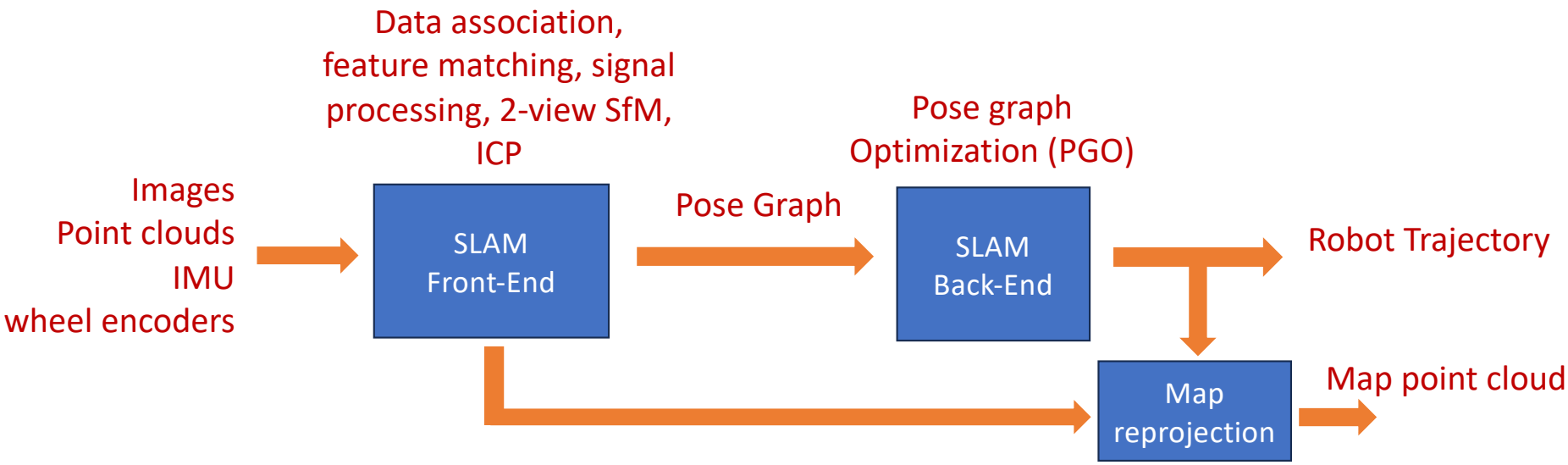
Johannes Schönberger, and collaborators, ETH Zurich





https://github.com/colmap/colmap

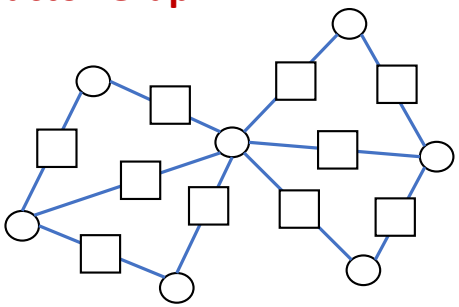
Pose Graph SLAM



Pose Graph SLAM - Produces robot traj first, map obtained in post processing.

Bundle Adjustment Optimization

Factor Graph:



Sum of all factors gives total cost:

Odometry factors (LIDAR, IMU/encoders)

$$J((R_i,\tau_i),q_k) = \sum_{(\hat{R}_{ij},\hat{\tau}_{ij})} (\lambda_{ij}^R \|R_i\hat{R}_{ij} - R_j\|^2 + \lambda_{ij}^\tau \|R_i\hat{\tau}_{ij} - (\tau_j - \tau_i)\|^2)$$

$$+ \sum_{\hat{q}_{ik}} \underbrace{\lambda_{ik}^D \|R_i\hat{q}_{ik} - (q_k - \tau_i)\|^2}_{\text{RGB-D factors}} + \sum_{(u_{ik},v_{ik})} \underbrace{\lambda_{ik}^V \|R_i K^{-1} p_{ik}^h - (q_k - \tau_i)\|^2}_{\text{RGB factors}}$$

Optimization Problem (Bundle Adjustment) :

$$\min_{(R_i,\tau_i),q_k} J((R_i,\tau_i),q_k)$$

$$\text{subject to} \quad \left.\begin{aligned} R_i^T R_i = I \\ \det(R_i) = 1 \end{aligned}\right\} \; SO(3)$$

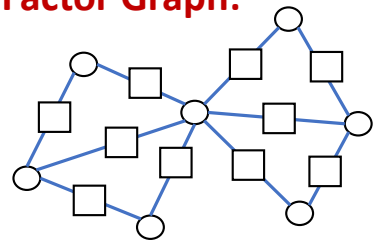
Interpretation: Find poses and feature locations that are *as consistent as possible* with all the measurements.

SLAM pose and map estimates:

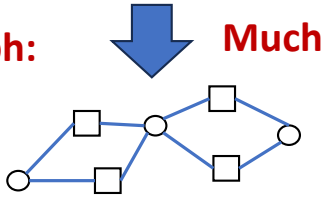
$$\{(R_i^*,\tau_i^*),q_k^*\} = \arg\min_{(R_i,\tau_i),q_k} J((R_i,\tau_i),q_k)$$

Pose Graph Optimization

Factor Graph:



Pose Graph:



Much simpler!

Interpretation: Still looking for poses *most consistent* with all relative pose measurements. Typically **much faster** than full bundle adjustment, but can be **less accurate**.

Pose Graph Cost Function:

Odometry factors (from all sensors)

$$J((R_i,\tau_i)_{i=1}^N) = \sum_{(\hat{R}_{ij},\hat{\tau}_{ij})} (\lambda_{ij}^R \|R_i\hat{R}_{ij} - R_j\|^2 + \lambda_{ij}^\tau \|R_i\hat{\tau}_{ij} - (\tau_j - \tau_i)\|^2)$$

Pose Graph Optimization (PGO):

$$\min_{(R_i,\tau_i)_{i=1}^N} J((R_i,\tau_i)_{i=1}^N)$$

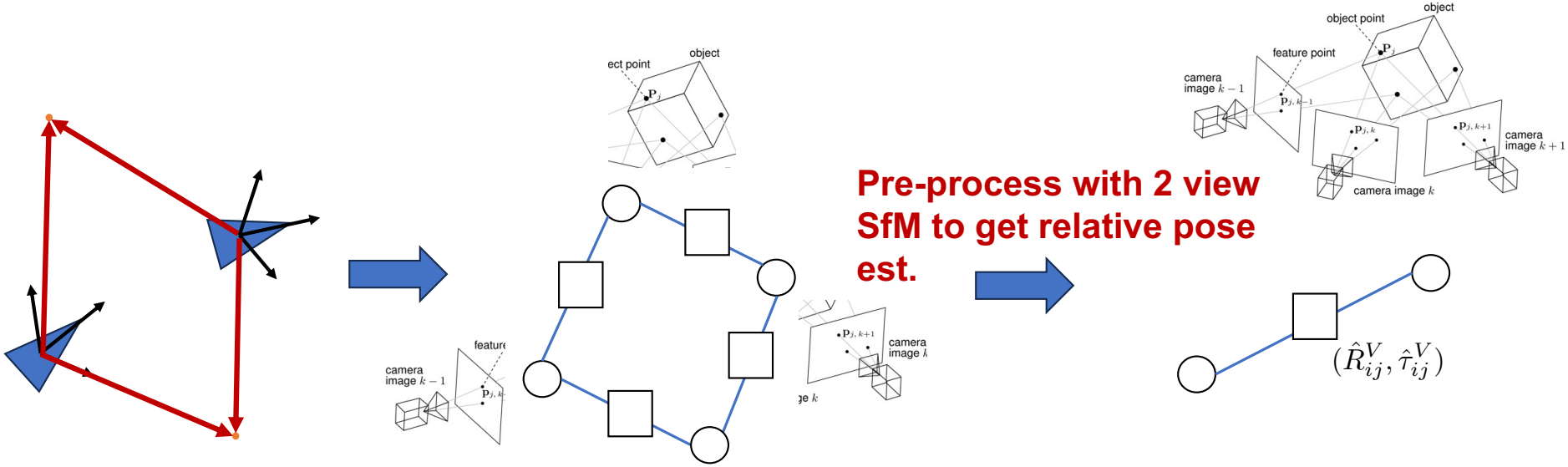
$$\text{subject to} \quad R_i^T R_i = I$$

$$\det(R_i) = 1$$

SLAM pose estimates only (no map!):

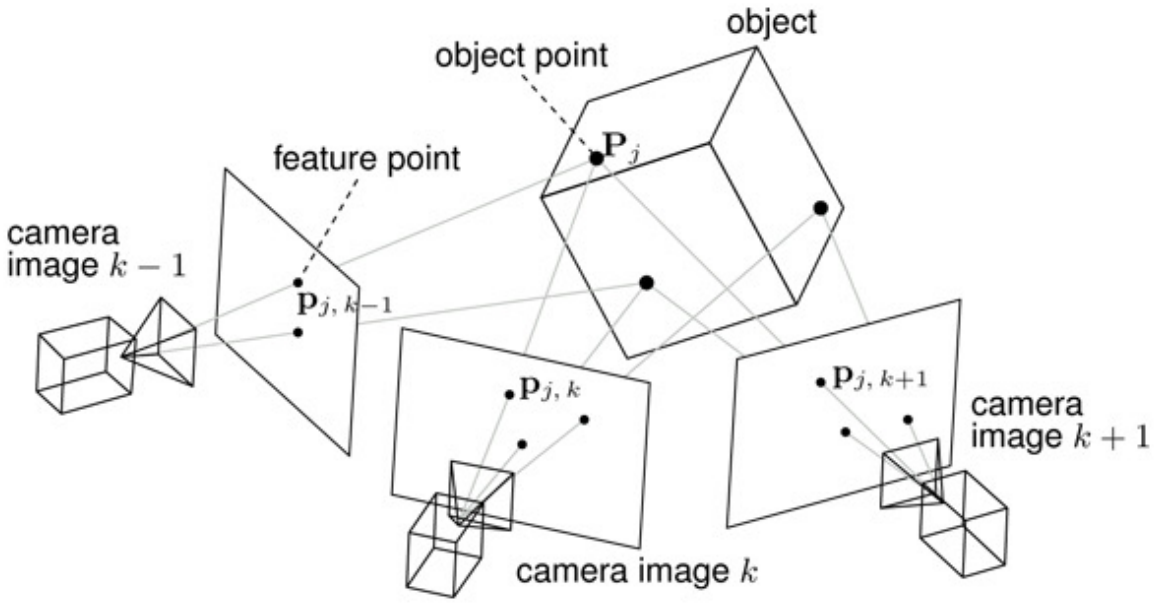
$$\{(R_i^*,\tau_i^*)_{i=1}^N\} = \arg\min_{(R_i,\tau_i)_{i=1}^N} J((R_i,\tau_i)_{i=1}^N)$$

SfM: Can reduce the camera-feature factor sub-graph to single odometry factor



Interpretation: Do more processing in front end to get a simpler optimization in the back end.

Structure from motion (SFM)

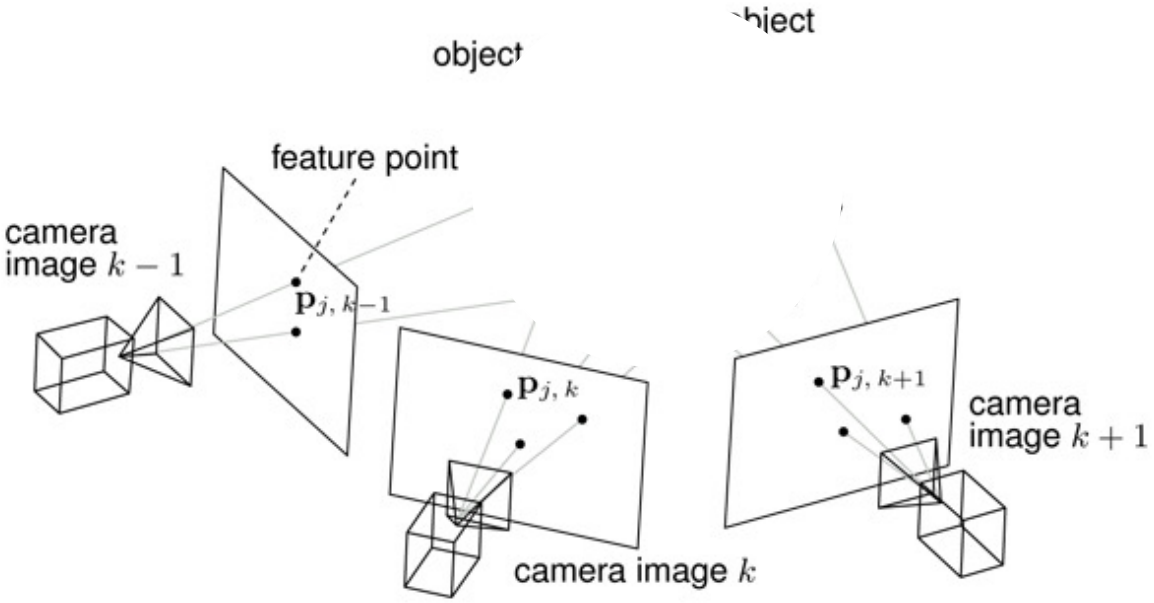


Given 2 or more images of *n* fixed 3D points

Find:

- Relative pose (motion) between 2 cameras
- Location of 3D points P_j (structure)

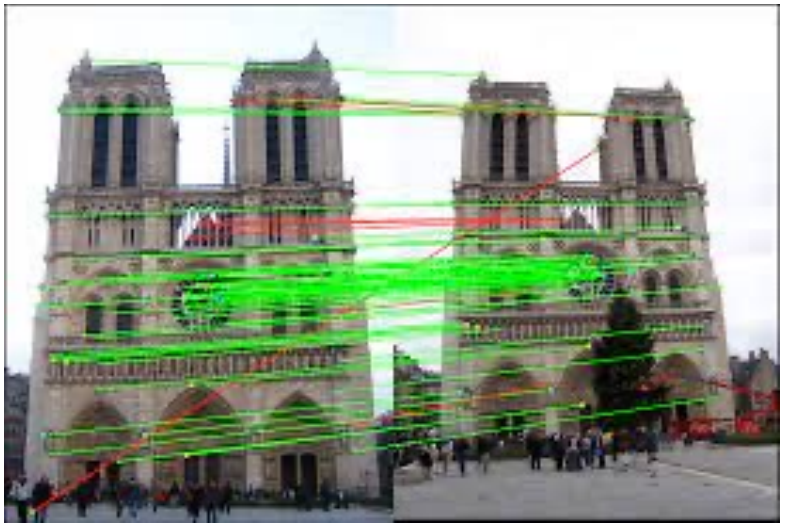
Feature matching



How do we know which pixels $p^h_{j,k}$ correspond to the same 3D point P_j ?
Without access to 3D info!

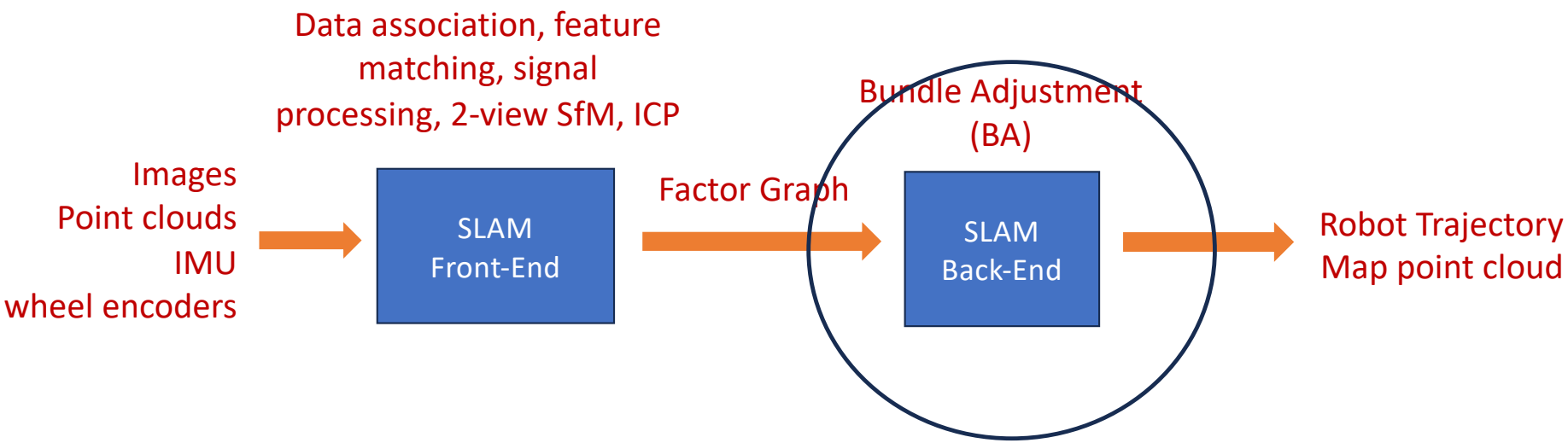
Random Sample Consensus (RANSAC)

- Input two images
- Feature detection, feature matching -> N correspondences
- Problem: in practice, we get loads of bad feature matches
- Use Random Sample Consensus (RANSAC) to reject outliers
- In RANSAC, we
  - Sample a small subset n of matched features
  - Solve SfM (8 point SVD-based algorithm) to get relative pose
  - Check solution fit with remaining (N-n) feature matches – record fit score
  - Repeat
  - Return the fit with the highest score



Factor Graphs: A Unifying Perspective for SLAM

(Frank Dellaert, 2012, 2017)



Bundle Adjustment SLAM - Produces both robot traj and map point cloud

Optimization techniques (e.g., iSAM, colmap)

Optimization Problem:

$$\min_{(R_i,\tau_i)_{i=1}^N,(q_k)_{k=1}^M} J((R_i,\tau_i)_{i=1}^N,(q_k)_{k=1}^M)$$

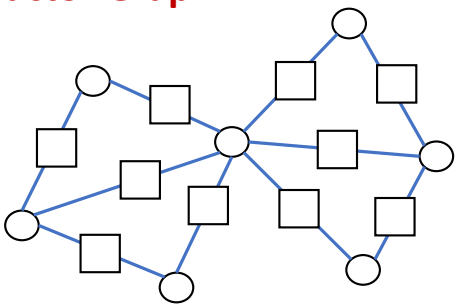
$$\text{subject to} \quad R_i^T R_i = I$$

$$\det(R_i) = 1$$

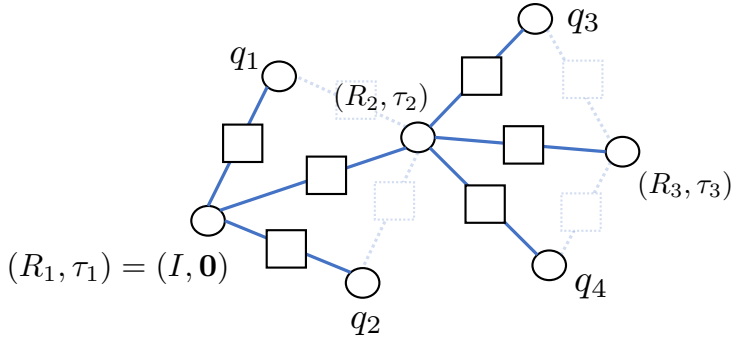
Large, non-convex, constrained problem.
In practice, many local minima.

Initialization: most optimization solvers require a rough initialization to converge to reasonable solution.

Factor Graph:



Initialization with min spanning tree:

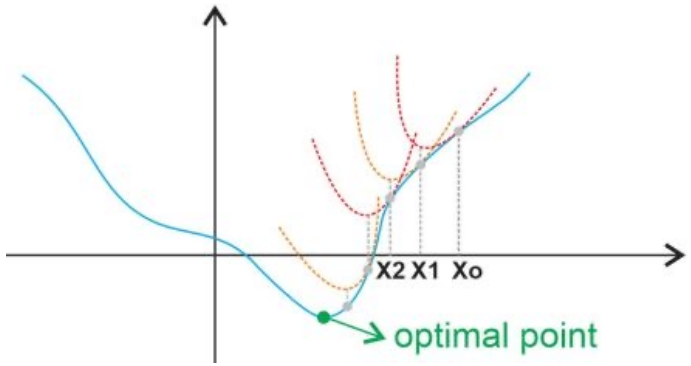


Optimization techniques (e.g., iSAM, colmap)

Optimization Problem:

$$\min_{(R_i,\tau_i)_{i=1}^N,(q_k)_{k=1}^M} J((R_i,\tau_i)_{i=1}^N,(q_k)_{k=1}^M)$$

$$\text{subject to} \quad R_i^T R_i = I$$
$$\det(R_i) = 1$$



Gauss-Newton Optimization

$$J = \sum_{(i,j)} f(T_i,T_j) = \sum_{(i,j)} \|l_{ij}(T_i,T_j)\|^2 \approx \sum_{(i,j)} \|l(T_i^0,T_j^0) + \nabla l(T_i^0,T_j^0)((T_i - T_j^0),(T_j - T_j^0))\|^2$$

Nonlinear least squares!

Linear least squares!

Taylor linearization about (T_i^0, T_j^0)

$$(T_i^1)_{i=1}^N = \arg\min \sum_{(i,j)} \|l(T_i,T_j) + \nabla l(T_i^0,T_j^0)((T_i - T_i^0),(T_j - T_j^0))\|^2$$

Iterate until convergence

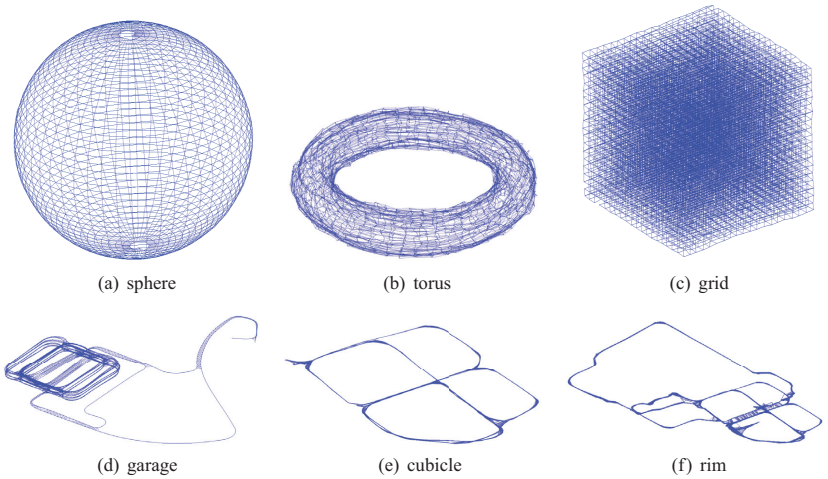
Levenberg-Marquart Optimization:

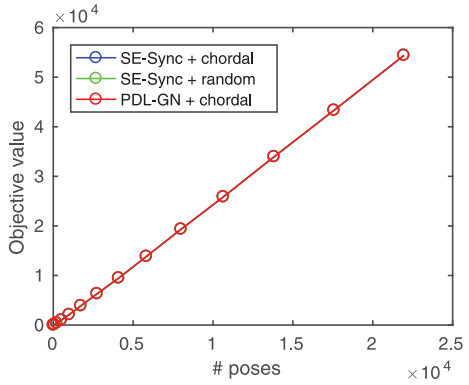
Combination of Gauss-Newton and Grad descent

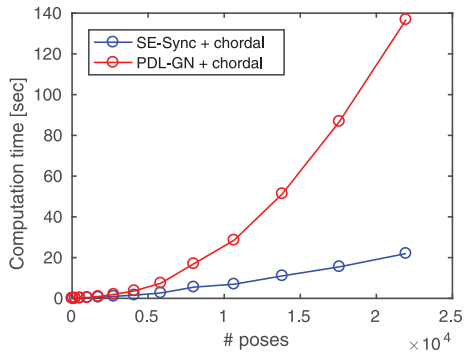
SE-Sync: Efficient Certifiable Solver for PGO

Rosen et al, IJRR 2018

Summary: Uses Riemannian optimization, duality, and semi-definite programming to solve PGO with suboptimality bound







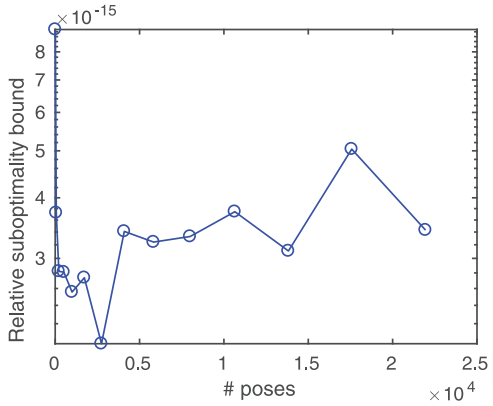


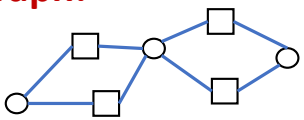
Fig. 5. Globally optimal solutions for the 3D SLAM benchmark datasets listed in Table 2.

Obtaining the Map from PGO solution

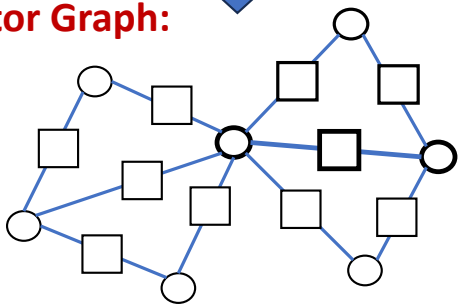
SLAM pose estimates:

$$\{(R_i^*, \tau_i^*)_{i=1}^N\} = \arg \min_{(R_i,\tau_i)_{i=1}^N} J((R_i, \tau_i)_{i=1}^N)$$

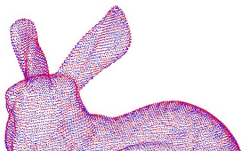
Pose Graph:

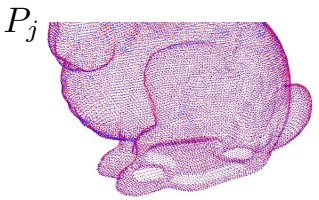


Factor Graph:



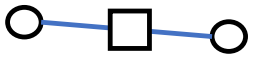
Lidar point clouds in local frames





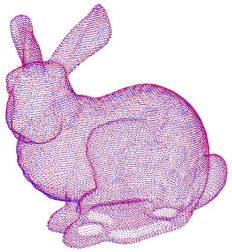
$P_i^* = R_i^* P_i + \tau_i^*$

$P_j^* = R_j^* P_j + \tau_j^*$



(R_i^*, τ_i^*) (R_j^*, τ_j^*)

Lidar point clouds in world frame



Fused point cloud in world frame

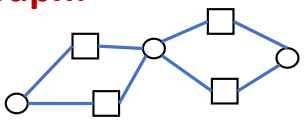
$\{P_i^* \cup P_j^*\}$

Obtaining the Map from PGO Solution

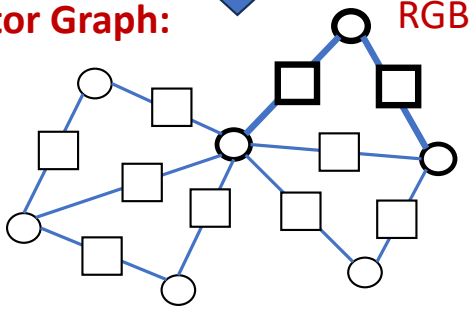
SLAM pose estimates:

$$\{(R_i^*, \tau_i^*)_{i=1}^N\} = \arg \min_{(R_i,\tau_i)_{i=1}^N} J((R_i, \tau_i)_{i=1}^N)$$

Pose Graph:

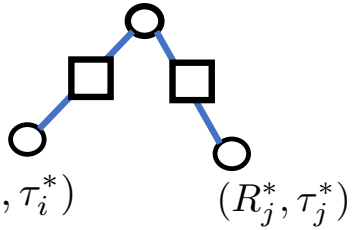


Factor Graph: RGB factors



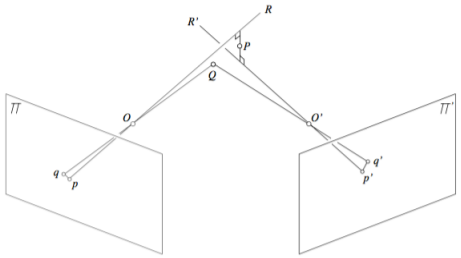
Point in point cloud

q_k^* unknown



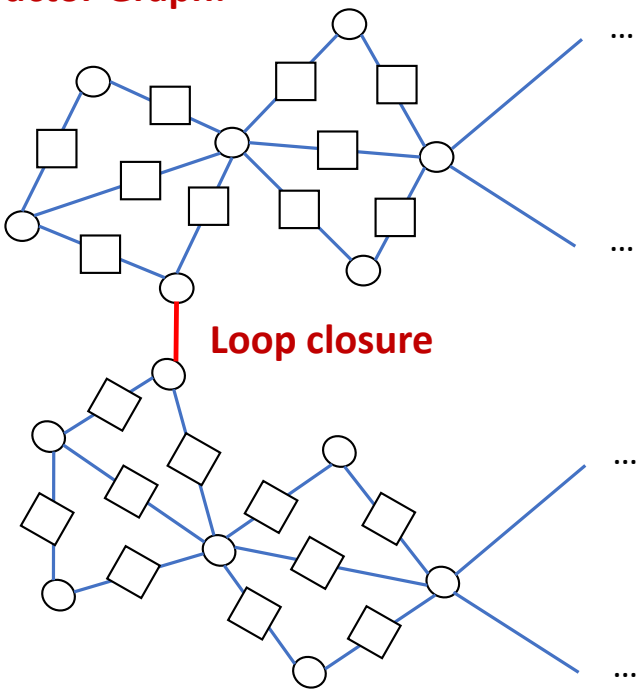
known

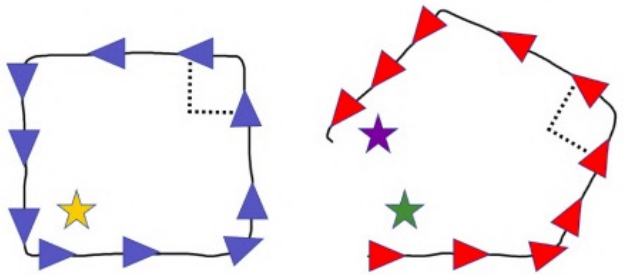
Stereo Depth: find location of point in 3D from two RGB images with known poses.



Loop Closure

Factor Graph:





- **Long time separation**
- **Close spatial separation**
- **Requires place recognition or special loop closure module**

Loop Closure

Factor Graph:

